GitHub repository: Deliverable 3 on GitHub

In [36]:
```
from character import Character, Warrior, Mage
from dice import Dice
from items import Weapon
from battle import Battle
```

TASK 1 - character

In [37]:
```
class Character:
    """Represents a character in the RPG"""

    # ----------------------------------------------------------------------
    # Task 3: __init__ accepts an optional weapon
    # ----------------------------------------------------------------------
    def __init__(self, name: str, hp_max: int = 100, damage: int = 10,
                 weapon: Weapon | None = None) -> None:
        """
        Initialise the Character

        Args:
            name (str): The name of the character.
            hp_max (int): Max hit points (default = 100)
            damage (int): Base damage dealt when attacking without a weap
            weapon (Weapon | None): Equipped weapon (default = None)
        """
        self.name = name
        self._hp = hp_max  # Note: _hp indicates private attribute
        self.hp_max = hp_max
        self.damage = damage
        self.weapon = weapon

    # ----------------------------------------------------------------------
    # Task 1a: take_damage()
    # ----------------------------------------------------------------------
    def take_damage(self, amount: int) -> None:
        """
        Reduce this character's own HP by `amount`.
        HP can never go below 0.

        Args:
            amount (int): Damage to take (must be >= 0)
        """
        if amount < 0:
            raise ValueError("Damage amount can not be negative")
        self._hp = max(self._hp - amount, 0)

    # ----------------------------------------------------------------------
    # Task 1b: heal()
```

```
    # ------------------------------------------------------------------
    def heal(self, amount: int) -> None:
        """
        Increase this character's own HP by `amount`.
        HP can never go above hp_max.

        Args:
            amount (int): HP to restore (must be >= 0)
        """
        if amount < 0:
            raise ValueError("Heal amount can not be negative")
        self._hp = min(self._hp + amount, self.hp_max)

    # ------------------------------------------------------------------
    # Task 1c + Task 3: attack() - uses take_damage(), and weapon if equi
    # ------------------------------------------------------------------
    def attack(self, target: "Character") -> None:
        """Attack target, using weapon damage if a weapon is equipped"""
        if self.weapon is not None:
            dmg = self.weapon.roll_damage()
        else:
            dmg = self.damage
        target.take_damage(dmg)

    # ------------------------------------------------------------------
    # Task 1d: is_alive (read-only property)
    # ------------------------------------------------------------------
    @property
    def is_alive(self) -> bool:
        """True if the character has more than 0 HP, otherwise False"""
        return self.hp > 0

    # ------------------------------------------------------------------
    # Unchanged from lecture (except the repr bug fix)
    # ------------------------------------------------------------------
    def introduce(self) -> None:
        """Prints a string that introduces the character"""
        print(f"Hi, my name is {self.name}!")

    def __repr__(self) -> str:
        """Representation of character object"""
        return f"<Character name={self.name} hp={self.hp} damage={self.da

    def __str__(self) -> str:
        """String representation of character object"""
        return f"Character: {self.name} with {self.hp} HP and {self.damag

    @property
    def hp(self) -> int:
        """Public read-only property to inspect HP without allowing direc
        return self._hp
```

- take_damage reduces the characters HP, men never below 0
- heal increases the charatcers HP, but never over hp_max
- is_alive returns "true" if HP is over 0

In [38]:
```python
# Quick tests for Task 1 – run with: python character.py
a = Character("Arne")
b = Character("Bjorg", hp_max=15)

a.attack(b)
print(b.hp, b.is_alive)

a.attack(b)
print(b.hp, b.is_alive)

b.heal(100)
print(b.hp)

try:
    b.is_alive = True
except AttributeError:
    print("OK: is_alive is read-only")

try:
    b.take_damage(-5)    # should fail, negative damage not allowed
except ValueError:
    print("OK: negative damage rejected")
```

```
5 True
0 False
15
OK: is_alive is read-only
OK: negative damage rejected
```

TASK 2 - Dice

In [39]:
```python
import random

class Dice:
    """Represents an n-sided dice (n >= 2)"""

    def __init__(self, sides: int = 6) -> None:
        """
        Initialise the Dice

        Args:
            sides (int): Number of sides on the dice (default = 6, must b
        """
        if sides < 2:
            raise ValueError("A dice must atleast have 2 sides")
        self.sides = sides

    # ----------------------------------------------------------------
```

```
    # Task 2a: roll()
    # ------------------------------------------------------------------
    def roll(self) -> int:
        """Roll the dice and return a random number between 1 and sides"""
        return random.randint(1, self.sides)

    # ------------------------------------------------------------------
    # Task 2b: __repr__
    # ------------------------------------------------------------------
    def __repr__(self) -> str:
        """Representation of dice object, e.g. Dice(sides=6)"""
        return f"Dice(sides={self.sides})"

    # ------------------------------------------------------------------
    # Task 2c: __call__ - makes d20() do the same as d20.roll()
    # ------------------------------------------------------------------
    def __call__(self) -> int:
        """Roll the dice when the object is called like a function"""
        return self.roll()
```

- Dice can have a number of sides
- roll returns a number between 1 and the number of sides

In [40]:
```
# Quick tests for Task 2 - run with: python dice.py
d6 = Dice()
d20 = Dice(sides=20)

print(repr(d6))
print(d20)
print(d6.roll())
print(d20())

# Roll many times and check that all results are within range
rolls = [d6.roll() for _ in range(1000)]
print(min(rolls), max(rolls))
print(sorted(set(rolls)))

try:
    Dice(sides=1)        # should fail, a dice needs at least 2 sides
except ValueError:
    print("OK: Dice with fewer than 2 sides rejected")
```

```
Dice(sides=6)
Dice(sides=20)
4
20
1 6
[1, 2, 3, 4, 5, 6]
OK: Dice with fewer than 2 sides rejected
```

TASK 3 - items

In [41]:
```python
class Weapon:
    """Represents a weapon that deals damage based on a dice roll"""

    def __init__(self, name: str, damage_dice: Dice) -> None:
        """
        Initialise the Weapon

        Args:
            name (str): The name of the weapon
            damage_dice (Dice): The dice used to calculate damage
        """
        self.name = name
        self.damage_dice = damage_dice

    def roll_damage(self) -> int:
        """Roll the weapon's damage dice and return the damage dealt"""
        return self.damage_dice.roll()

    def __repr__(self) -> str:
        """Representation of weapon object, e.g. Weapon(name=Sword, damag
        return f"Weapon(name={self.name}, damage_dice={self.damage_dice})
```

- creates a weapon and gives it a name
- roll_damage rolls the weapon's dice and returns the damage

In [42]:
```python
# Quick tests for Task 3 - run with: python items.py
sword = Weapon("Sword", Dice(sides=8))

print(sword)
print(sword.roll_damage())

rolls = [sword.roll_damage() for _ in range(1000)]
print(min(rolls), max(rolls))
```

```
Weapon(name=Sword, damage_dice=Dice(sides=8))
2
1 8
```

In [43]:
```python
# Tests for Task 3
from dice import Dice

sword = Weapon("Sword", Dice(sides=8))
c = Character("Cecilie", weapon=sword)
d = Character("Dag")

c.attack(d)
print(d.hp)

d.attack(c)
print(c.hp)
```

```
99
90
```

TASK 4 - warrior subclass

In [44]:

```python
# ----------------------------------------------------------------------
# Task 4a: Warrior subclass (defensive specialist)
# ----------------------------------------------------------------------
class Warrior(Character):
    """A defensive character whose armor reduces incoming damage"""

    def __init__(self, name: str, hp_max: int = 100, damage: int = 10,
                 weapon: Weapon | None = None, armor: int = 5) -> None:
        """
        Initialise the Warrior

        Args:
            name, hp_max, damage, weapon: Same as Character
            armor (int): Reduces all incoming damage by this amount (defa
        """
        super().__init__(name, hp_max, damage, weapon)
        self.armor = armor

    def take_damage(self, amount: int) -> None:
        """Reduce incoming damage by armor before applying it (never belo
        if amount < 0:
            raise ValueError("Damage amount can not be negative")
        reduced = max(amount - self.armor, 0)
        super().take_damage(reduced)

#----------------------------------------------------------------------
#Task 4b: Mage subclass (resocurce based attacker)
#----------------------------------------------------------------------
class Mage(Character):
    """A magic user who spends mana on powerful spell attacks"""

    SPELL_COST = 10     # mana needed for one spell
    MANA_REGEN = 5      # mana restored after a base attack

    def __init__(self, name: str, hp_max: int = 100, damage: int = 10,
                 weapon: Weapon | None = None, mana_max: int = 30) -> Non
        """
        Initialise the Mage

        Args:
            name, hp_max, damage, weapon: Same as Character
            mana_max (int): Maximum mana (default = 30). The mage starts
        """
        super().__init__(name, hp_max, damage, weapon)
        self.mana_max = mana_max
        self.mana = mana_max

    def attack(self, target: "Character") -> None:
```

```
        """
        Cast a spell if enough mana, otherwise do a base attack.

        - mana >= 10: spend 10 mana, deal double base damage (ignores wea
        - mana < 10: normal attack (uses weapon if equipped), restore 5 m
        """
        if self.mana >= self.SPELL_COST:
            self.mana -= self.SPELL_COST
            target.take_damage(self.damage * 2)
        else:
            super().attack(target)
            self.mana = min(self.mana + self.MANA_REGEN, self.mana_max)
```

- creates two subclasses
- one with armor that reduces every hit with 5 HP
- the other can cast a spell using MANA.
- Every spell requires 10 MANA, else the character will do a basic attack

In [45]:
```
# Tests for Task 4a
w = Warrior("Wenche")
a.attack(w)
print(w.hp)

weak = Character("Weak", damage=3)
weak.attack(w)          # 3 damage, minus 5 armor -> 0
print(w.hp)
```

```
95
95
```

In [46]:
```
# Tests for Task 4b
m = Mage("Mia")
t = Character("Target")
for i in range(6):
    m.attack(t)
    print(f"Attack {i + 1}: target hp={t.hp}, mana={m.mana}")

staff = Weapon("Staff", Dice(sides=4))
m2 = Mage("Geir Eskil", weapon=staff, mana_max=10)
t2 = Character("Target2")
m2.attack(t2)
print(t2.hp)
m2.attack(t2)
print(t2.hp)
```

```
Attack 1: target hp=80, mana=20
Attack 2: target hp=60, mana=10
Attack 3: target hp=40, mana=0
Attack 4: target hp=30, mana=5
Attack 5: target hp=20, mana=10
Attack 6: target hp=0, mana=0
80
76
```

TASK 5 - batte

In [47]:
```python
class Battle:
    """Manages turn-based combat between two characters"""

    def __init__(self, hero: Character, enemy: Character) -> None:
        """Initialise a battle between two characters"""
        self.hero = hero
        self.enemy = enemy

    def play_round(self) -> None:
        """Play one round: hero attacks first, then enemy attacks if stil
        self.hero.attack(self.enemy)
        if self.enemy.is_alive:
            self.enemy.attack(self.hero)

    def simulate_battle(self) -> Character:
        """Run rounds until one character is no longer alive, and return
        while self.hero.is_alive and self.enemy.is_alive:
            self.play_round()
        return self.hero if self.hero.is_alive else self.enemy
```

- Battle with a hero and a enemy
- the hero attacks first, and if the enemy is still alive, the enemy attacks back
- `simulate_battle()` : runs rounds in a loop until one character is no longer alive, and returns the winner

We run one battle with weapons and one without weapons.

In [48]:
```python
# Quick tests for Task 5 — run with: python battle.py
from character import Warrior, Mage
from dice import Dice
from items import Weapon

hero = Warrior("Wenche", weapon=Weapon("Sword", Dice(sides=12)))
enemy = Mage("Geir Eskil")

winner = Battle(hero, enemy).simulate_battle()
print(f"Winner: {winner.name}")
print(f"{hero.name}: {hero.hp} HP, {enemy.name}: {enemy.hp} HP")
# expected: one of them has 0 HP, the other has more than 0
```

```
    # Battle without weapons
hero2 = Warrior("Jørgen")
enemy2 = Mage("Markus")
winner2 = Battle(hero2, enemy2).simulate_battle()
print(f"Without weapons - Winner: {winner2.name}")
print(f"{hero2.name}: {hero2.hp} HP, {enemy2.name}: {enemy2.hp} HP")
```

```
Winner: Geir Eskil
Wenche: 0 HP, Geir Eskil: 32 HP
Without weapons - Winner: Jørgen
Jørgen: 5 HP, Markus: 0 HP
```

Use of GenAI

We used Claude (Anthropic) as a tutor throughout Tasks 1–5, mainly for guidance and error fixing. We asked it to explain the assignment text and Python concepts such as `__init__`, properties, inheritance, `super()`, `__repr__` and composition. When our code did not work, we used it to find and understand the errors (e.g. indentation, wrong variables, missing imports). For some parts it suggested code. We did not use these suggestions blindly: we went through them, adapted them to our own code, and asked follow-up questions until we understood every line. We wrote the `Battle` class ourselves, and all code was run and tested by us. Design choices, such as how to interpret "double damage" and which dice to use for weapons, were discussed and decided by the group.